SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Experiments

Course Code:	ITA0517	**Course Title:**	Computer Vision
CO Assessed:	CO3 – Precision (accurate feature extraction & analysis) (BL3)	**Assessment:**	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	**Total Marks:**	100
CO3 Statement:	*Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)*		
Student Name:	Yoga Madha A	**Reg. No.:**	192425058
Section / Batch:	B	**Date:**	12-08-2026

Code of Conduct

I **Yoga Madha A-192425058** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: 

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts	
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation	
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools	
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results	
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation	

1. Effect of Smoothing on Edge Detection

Design and perform an experiment to study the impact of smoothing techniques on edge detection performance. Explain the theoretical purpose of smoothing in reducing noise and improving edge continuity. Apply smoothing and edge detection methods using appropriate tools, document outputs before and after smoothing systematically, and analyze how smoothing affects edge clarity, continuity, and noise suppression.

Answer:

Aim

To design and perform an experiment to study the effect of smoothing techniques on edge detection performance, and to analyze how smoothing reduces noise, improves edge continuity, and affects edge clarity.

Problem Statement

Noise and small intensity variations in an image can produce unwanted edges during edge detection. Smoothing is therefore applied before edge detection to reduce noise and improve the detection of meaningful object boundaries. This experiment compares edge detection **before and after Gaussian smoothing** and analyzes the effect on edge quality.

Objectives

1. To understand the theoretical purpose of image smoothing.
2. To reduce noise using Gaussian filtering.
3. To perform edge detection using the Canny edge detector.
4. To compare edge detection results before and after smoothing.
5. To analyze edge clarity, continuity, noise suppression, and loss of fine details.
6. To identify a suitable smoothing level for effective edge detection.

Theory

Smoothing

Smoothing is an image-processing technique used to reduce noise, small intensity variations, and unwanted details from an image. It works by averaging or weighting neighboring pixels.

Noise can create sudden changes in pixel intensity. Since edge detectors identify sudden intensity changes, noise can be incorrectly detected as edges. Applying smoothing before edge detection reduces these unwanted variations.

The general process is:

Input Image → Grayscale Conversion → Smoothing → Edge Detection → Result Analysis

Gaussian Smoothing: In this experiment, a **Gaussian filter** is used for smoothing. Gaussian filtering calculates a weighted average of neighboring pixels. Pixels closer to the center of the filter have greater importance.

The Gaussian function is:

$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

where:

- (x,y) = pixel coordinates
- (\sigma) = standard deviation
- (G(x,y)) = Gaussian weight

A small kernel provides mild smoothing, while a larger kernel provides stronger smoothing.

Edge Detection

Edge detection identifies boundaries where there is a significant change in intensity. In this experiment, the **Canny edge detector** is used.

Canny edge detection consists of several important stages:

1. Noise reduction
2. Gradient calculation
3. Non-maximum suppression
4. Double thresholding
5. Edge tracking by hysteresis

Canny produces a binary edge image in which detected edges are represented by bright pixels.

Experimental Design

To satisfy the experimental-design criterion, the same image and same Canny parameters are used for all experiments.

Three conditions are compared:

Experiment	**Smoothing**	**Purpose**
Experiment 1	No smoothing	Establish baseline
Experiment 2	Gaussian 3×3	Mild noise reduction
Experiment 3	Gaussian 5×5	Moderate smoothing
Experiment 4	Gaussian 7×7	Strong smoothing

The Canny thresholds are kept constant at **100 and 200** so that the effect of smoothing can be observed fairly.

Algorithm

1. Start the program.
2. Read the input image.
3. Check whether the image was loaded successfully.
4. Convert the image from RGB/BGR to grayscale.
5. Apply Canny edge detection directly to the grayscale image.
6. Apply Gaussian smoothing using a **3×3 kernel**.
7. Apply Canny edge detection to the smoothed image.
8. Apply Gaussian smoothing using a **5×5 kernel**.
9. Apply Canny edge detection again.
10. Apply Gaussian smoothing using a **7×7 kernel**.
11. Apply Canny edge detection again.
12. Display all results.
13. Compare noise, edge clarity, continuity, and fine-detail preservation.
14. Determine the most suitable smoothing level.
15. Stop.

Tools and Technologies

- **Programming Language:** Python
- **Image Processing Library:** OpenCV
- **Visualization Library:** Matplotlib
- **Edge Detection:** Canny
- **Smoothing Technique:** Gaussian filtering
- **Development Environment:** Python IDLE / VS Code / Google Colab

Python Code

```
import cv2
import matplotlib.pyplot as plt

# Read input image
image = cv2.imread("input.jpg")

# Check whether image is loaded
if image is None:
    print("Error: input.jpg not found.")
    print("Keep the image in the same folder as this Python file.")
```

```
    exit()

# Convert image to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# 1. Edge Detection WITHOUT Smoothing
edges_original = cv2.Canny(gray, 100, 200)

# 2. Gaussian Smoothing - 3x3
smooth_3 = cv2.GaussianBlur(gray, (3, 3), 0)
edges_3 = cv2.Canny(smooth_3, 100, 200)

# 3. Gaussian Smoothing - 5x5
smooth_5 = cv2.GaussianBlur(gray, (5, 5), 0)
edges_5 = cv2.Canny(smooth_5, 100, 200)
# 4. Gaussian Smoothing - 7x7
smooth_7 = cv2.GaussianBlur(gray, (7, 7), 0)
edges_7 = cv2.Canny(smooth_7, 100, 200)

# Display Results
plt.figure(figsize=(12, 10))

plt.subplot(3, 2, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

plt.subplot(3, 2, 2)
plt.imshow(edges_original, cmap="gray")
plt.title("Edges - No Smoothing")
plt.axis("off")

plt.subplot(3, 2, 3)
plt.imshow(smooth_3, cmap="gray")
plt.title("Gaussian Smoothing 3x3")
plt.axis("off")

plt.subplot(3, 2, 4)
plt.imshow(edges_3, cmap="gray")
plt.title("Edges After 3x3 Smoothing")
plt.axis("off")

plt.subplot(3, 2, 5)
plt.imshow(smooth_5, cmap="gray")
plt.title("Gaussian Smoothing 5x5")
plt.axis("off")
```

```
plt.subplot(3, 2, 6)
plt.imshow(edges_5, cmap="gray")
plt.title("Edges After 5x5 Smoothing")
plt.axis("off")

plt.tight_layout()
plt.show()

# Second figure for 7x7 result
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(smooth_7, cmap="gray")
plt.title("Gaussian Smoothing 7x7")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(edges_7, cmap="gray")
plt.title("Edges After 7x7 Smoothing")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Expected Output

Original Image

Edges - No Smoothing

Gaussian Smoothing 3x3

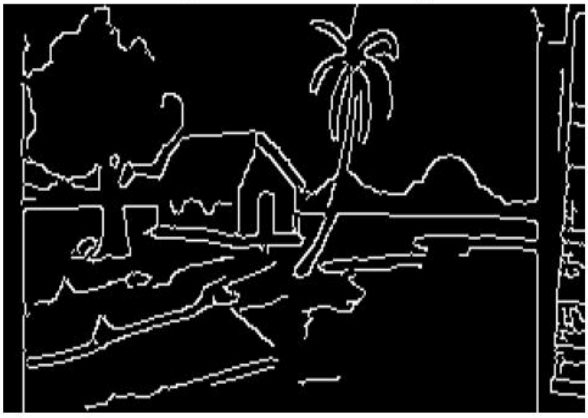
Edges After 3x3 Smoothing

Gaussian Smoothing 5x5

Edges After 5x5 Smoothing

Gaussian Smoothing 7x7

Edges After 7x7 Smoothing



Output Flow

Original Image

↓

Grayscale Image

↓

Canny without smoothing

↓

Gaussian 3×3 → Canny

↓

Gaussian 5×5 → Canny

↓

Gaussian 7×7 → Canny

↓

Comparison and Analysis

Observation

Smoothing Level	**Noise Suppression**	**Edge Continuity**	**Edge Sharpness**	**Fine Details**
No smoothing	Low	Lower in noisy areas	High but noisy	High
3×3	Moderate	Improved	Good	Mostly preserved
5×5	Good	Good	Slightly reduced	Some small details removed
7×7	Very high	Smooth	Lower	More details lost

Detailed Analysis

1. Without Smoothing

When Canny edge detection is applied directly to the image, noise and small intensity variations may be detected as edges. Therefore, the output can contain several unwanted or false edges.

The edges may be sharp, but the result can be noisy and less reliable.

2. 3×3 Gaussian Smoothing

The 3×3 Gaussian filter provides mild smoothing. It removes some small noise while preserving most important image structures.

The detected edges become cleaner while maintaining good sharpness.

3. 5×5 Gaussian Smoothing

The 5×5 filter provides stronger noise suppression. Unwanted small edges are reduced, and major object boundaries become more continuous.

This usually provides a good balance between:

Noise Reduction + Edge Preservation

Therefore, the 5×5 filter can be considered a suitable choice for moderate noise.

4. 7×7 Gaussian Smoothing

The 7×7 filter performs stronger smoothing. More noise is removed, but important fine details may also disappear.

Edges become smoother but less sharp. Small objects and narrow boundaries may not be detected as clearly.

Comparison of Edge Detection

Before Smoothing

- More unwanted edges may appear.
- Noise can be interpreted as edges.
- Edge continuity can be poor.
- Fine details are preserved.
- The result may look cluttered.

After Smoothing

- Noise is reduced.
- False edges are reduced.
- Major boundaries become more continuous.
- Edge detection becomes more stable.
- Excessive smoothing can blur important boundaries.

Analysis According to the Experiment

The experiment demonstrates that smoothing has a direct effect on edge-detection performance.

When no smoothing is applied, Canny detects both meaningful boundaries and intensity variations caused by noise. Therefore, the edge image can contain unwanted edges.

When a small amount of Gaussian smoothing is applied, noise is reduced while most important edges remain visible. With moderate smoothing, such as **5×5**, the edges become cleaner and more continuous.

When strong smoothing, such as **7×7**, is applied, noise suppression increases further, but some fine details and weak edges may be lost.

Therefore, smoothing involves a trade-off:

More Smoothing→Less Noise→More Blurring

and

Less Smoothing→More Detail→More Noise

Validation of Results

The results can be validated by comparing the edge images using the following factors:

Noise suppression:
The number of unwanted small edges should decrease after smoothing.

Edge continuity:
Important object boundaries should appear more connected after moderate smoothing.

Edge clarity:
Moderate smoothing should produce cleaner edges, while excessive smoothing may make edges less sharp.

Fine-detail preservation:
Small details are better preserved with little or no smoothing but may disappear with strong smoothing.

Thus, the visual comparison of the four edge maps validates the effect of different smoothing levels.

Result

The experiment was successfully designed and performed to study the **effect of smoothing on edge detection**. Gaussian smoothing reduced image noise and unwanted edges and improved the continuity of important object boundaries. Among the tested filters, **moderate smoothing such as a 5×5 Gaussian filter provides a good balance between noise suppression and edge preservation**. Excessive smoothing, such as 7×7, reduces noise further but can blur edges and remove fine details.

Conclusion

Smoothing is an important preprocessing step for reliable edge detection. It reduces unwanted intensity variations before the Canny detector identifies boundaries. The experiment confirms that **appropriate smoothing improves edge continuity and noise suppression**, but excessive smoothing can reduce edge sharpness and eliminate useful image details.